

UATimer: Lightweight Context-Aware Power Management for Heterogeneous IoT Systems

Geunsik Lim

Abstract—Energy-constrained Internet-of-Things (IoT) devices must enter low-power states aggressively without imposing unacceptable wake-up delay. Fixed inactivity timeouts are inexpensive but insensitive to changing activity, while predictive policies can add training, inference, and model-management overhead. This paper presents UATimer, a lightweight context-aware adaptive timer that updates an idle threshold from observed inter-event intervals and shifts energy–latency priorities using coarse application context. The framework runs in user space, uses standard timer and event interfaces, requires fewer than 500 lines of code and less than 1 KB of policy state, and does not require kernel modification. We evaluate the mechanism on a smartphone, a tablet, and an IoT sensor node using web browsing, video playback, on-device AI inference, AR/VR gaming, and periodic sensing workloads. Across the reported workload means, UATimer reduces energy by 20.5–37.5% relative to the default baseline and by 7.9–16.7% relative to a fixed-timeout policy while keeping average event delay below 50 ms. In the AR/VR workload, it reports a 0.5% frame-drop rate and 1.2 ms jitter. Against a deliberately lightweight linear-regression predictor, UATimer yields similar energy values with a smaller policy footprint and no trained model. The results support UATimer as a deployable policy layer between static timeouts and heavier predictive power-management approaches.

Index Terms—Internet of Things, constrained devices, power management, energy efficiency, adaptive timer, context-aware systems, embedded systems.

I. INTRODUCTION

BATTERY-operated IoT and mobile-edge platforms increasingly execute interactive, sensing, communication, and inference workloads on the same device. Their energy budget is constrained not only by computation but also by repeated transitions among active, idle, and low-power states. A timeout that is too long wastes energy during idle intervals, whereas an overly aggressive timeout increases wake-up frequency and response delay. This tension is particularly important for systems that mix latency-sensitive tasks with long background idle periods.

Conventional operating-system power policies typically rely on fixed inactivity thresholds or coarse state machines. Such approaches are attractive because they are deterministic and inexpensive, but they can be poorly aligned with actual user behavior. Network-assisted duty cycling and IoT power-saving modes reduce active time, but their energy–latency behavior depends strongly on radio and paging configuration [5], [6]. Mobile-system studies likewise show that idle and suspended states materially affect device energy use [1]. These mechanisms do not directly solve the application-level question of

how an interactive endpoint should adapt its suspend threshold to changing activity.

Recent work has explored context-aware mobile scheduling [4], predictive sleep scheduling [6], and learning-based sleep policies for energy-constrained IoT systems [8], [10]. However, the cost of adaptation can be non-trivial for resource-constrained devices. For small embedded platforms, even lightweight models introduce additional state, inference logic, and model-management complexity. This motivates a design point that is more adaptive than a fixed timeout but substantially simpler than a predictive learning system.

This paper develops such a design through a *user-aware adaptive timer*. The framework observes inter-event idle periods, updates the timeout online, and modifies the energy–latency preference according to lightweight context. It is implemented as a user-space component that interoperates with existing Linux/Android power-management stacks and standard timer APIs. The design targets heterogeneous platforms ranging from smartphones and tablets to low-power IoT sensor nodes.

The main contributions are as follows:

- We present a deployable user-space power-management layer that separates a portable suspend/resume mechanism from an online timeout policy, avoiding kernel modification and model training.
- We use a constant-state exponentially weighted idle-history update together with explicit context-dependent energy–delay priorities; the smoothing coefficient and objective weights are defined separately to avoid conflating temporal adaptation with policy preference.
- We describe an implementation with fewer than 500 lines of code and less than 1 KB of policy state, using PowerHAL-related callbacks on Android and `epoll`-based event hooks on Linux.
- We evaluate the mechanism across smartphone, tablet, and IoT-node platforms and five representative workloads, reporting energy, latency, and AR/VR quality indicators while explicitly identifying the limited IoT-platform breadth and five-run sample size.
- We compare against default OS behavior, fixed timeouts, and a deliberately lightweight linear-regression predictor. The comparison is positioned as evidence for a low-complexity operating point, not as a claim of superiority over modern TinyML or learning-based power managers.

The remainder of this paper is organized as follows. Section II summarizes the design motivation and related approaches. Section III presents the system architecture and adaptation algorithm. Section IV describes the implementation. Section V explains the experimental methodology, and

G. Lim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, 2066 Seobu-ro, Jangsan-gu, Suwon, Republic of Korea (e-mail: leemgs@g.skku.edu).

Section VI reports the results. Section VII discusses implications, limitations, and extensions, followed by the conclusion in Section VIII.

II. BACKGROUND AND DESIGN MOTIVATION

A. Idle-State Power Management

Mobile and IoT systems spend substantial time waiting for user input, network packets, sensor events, or periodic timers. Entering a deeper low-power state during these intervals can reduce energy consumption, but the transition itself carries latency and energy costs. Therefore, an effective policy must decide not only *whether* to suspend but also *when* to do so.

Fixed inactivity timeouts are commonly used because their behavior is predictable. Prior mobile work has shown that user context can expose idle intervals suitable for energy saving [2], [3], while empirical NB-IoT studies show that power-state and paging configuration materially affect device energy consumption [5]. The drawback is that a fixed threshold cannot distinguish a bursty interactive session from a long unattended interval. A threshold chosen for responsiveness may remain too conservative at night or during sparse sensing, while a threshold optimized for energy saving can be disruptive during an AR/VR session.

B. Learning-Based Policies

Learning-based schedulers attempt to predict future idle intervals or workload states. In principle, prediction can reduce unnecessary transitions and increase energy savings. In practice, learning introduces new implementation requirements: feature extraction, model state, inference, training or retraining, and potentially hardware acceleration. These costs may be acceptable on high-end systems but less attractive for small IoT nodes or products that prioritize deterministic behavior and implementation simplicity.

The proposed framework is deliberately positioned between these two extremes. It uses recent observed idle intervals to adapt the timeout and allows simple context rules to adjust priorities. No training phase is required, and the policy state can be represented by a small number of scalar variables.

C. Design Goals

The framework was designed around five goals:

- 1) **Low overhead:** negligible CPU and memory cost in comparison with the managed workload.
- 2) **Responsiveness:** wake-up delay should remain suitable for interactive use.
- 3) **Adaptivity:** the timeout should follow changing idle behavior rather than remaining fixed.
- 4) **Portability:** the implementation should depend on generic user-space timer and event mechanisms rather than ISA-specific optimizations.
- 5) **Deployability:** integration should not require kernel or driver modifications.

TABLE I
POSITIONING OF UATIMER RELATIVE TO COMMON TECHNIQUES

Technique	Primary control variable	Typical strength	Limitation addressed by UATimer
Fixed timeout	Idle duration	Simplicity, determinism	No workload adaptation
DVFS/task mapping	Voltage/frequency, placement	Compute efficiency	Does not decide idle suspend timing
Protocol duty cycle	Radio/sampling schedule	Communication energy	Often schedule-driven
ML scheduler	Predicted idle/workload state	Rich adaptation	Training/inference complexity
UATimer	Adaptive threshold	idle Low-cost online adaptation	Coarse rather than learned context

D. Relation to Existing Power-Management Techniques

The proposed timer is intended to complement, rather than replace, established platform mechanisms. DVFS controls the energy cost of computation by changing voltage and frequency, while timer-based suspend/resume controls whether the platform should remain active during an idle interval. Network-side mechanisms such as duty cycling and predictive sleep scheduling reduce communication activity according to protocol behavior or learned delivery timing [6], [10]. UATimer operates at a different layer: it uses observed application and user activity to decide when a generic low-power transition is worthwhile. Consequently, the framework can coexist with processor-level DVFS and radio-specific power-saving mechanisms.

This distinction is also relevant to learning-based schedulers. A predictor attempts to estimate future behavior before making a transition decision. UATimer instead reacts to measured idle history and a small set of context labels. The latter cannot exploit complex latent correlations that might be available to a richer model, but it avoids training data, model distribution, inference dependencies, and retraining when the workload mix changes. For resource-constrained IoT nodes, eliminating these dependencies can be a meaningful engineering advantage even when the memory footprint of an individual model is small.

E. IoT-Oriented Design Requirements

An IoT-oriented policy must operate under constraints that are less visible on server-class systems. First, policy state should remain small enough to coexist with application buffers and protocol stacks on memory-constrained nodes. Second, adaptation should not require a persistent training pipeline, because many deployed devices have intermittent connectivity and long maintenance cycles. Third, the control path should be inspectable: a developer should be able to determine why a particular timeout was selected and to impose platform-specific minimum or maximum values. Finally, the same policy should remain usable when the endpoint changes from an interactive mobile device to a sensing node.

UATimer follows these requirements by maintaining only the current threshold, recent timing information, and a small amount of context state. The update in Eq. (1) is independent of the processor instruction set and does not assume a specific radio technology. The power-controller interface is therefore

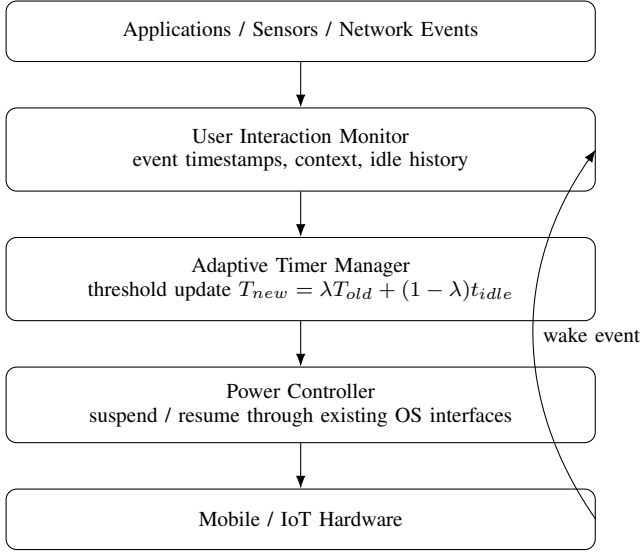


Fig. 1. Architecture of the user-aware adaptive timer framework.

the platform-specific boundary: Android can map a transition to existing power-management services, Linux can use its suspend interfaces, and an MCU-class platform can map the same decision to its supported sleep state. This separation is central to the heterogeneous-system objective of the design.

F. Parameterization Strategy

The design uses representative energy-conservative, balanced, and latency-critical weight settings rather than a single universal configuration. This is appropriate because the acceptable latency penalty is application-dependent. A practical configuration process can therefore proceed in two stages. The platform first selects safe bounds T_{min} and T_{max} from its wake-up characteristics. The application then selects a context mode that determines the relative emphasis in Eq. (2). Within those limits, Eq. (1) continuously adapts to observed idle behavior.

This decomposition keeps parameter tuning interpretable. The smoothing factor λ determines temporal responsiveness, the threshold bounds encode platform constraints, and (w_E, w_D) encode application preference. None of these quantities requires a learned model. If future implementations introduce TinyML or statistical prediction, the predictor can be used to propose t_{idle} while retaining the same bounded power-control interface.

III. USER-AWARE ADAPTIVE TIMER FRAMEWORK

A. System Architecture

Fig. 1 shows the architecture. The *User Interaction Monitor* captures user input and background events. The *Timer Manager* maintains and updates the idle threshold. The *Power Controller* invokes the underlying low-power transition and resumes the system on a new event.

The framework does not alter the kernel scheduler or device drivers. Instead, it observes activity in user space and uses existing power-management interfaces. This separation

TABLE II
REPRESENTATIVE ENERGY-DELAY WEIGHT SETTINGS

w_E	w_D	Priority mode
0.8	0.2	Energy-conservative
0.5	0.5	Balanced
0.2	0.8	Latency-critical

minimizes integration effort and allows the same control logic to be reused across different processor architectures.

B. Adaptive Timer Update

Let T denote the current idle threshold. When no activity occurs for T , the device enters a low-power state. When the next event occurs, the framework records the observed idle interval t_{idle} and updates the threshold as

$$T_{new} = \lambda T_{old} + (1 - \lambda)t_{idle}, \quad (1)$$

where $0 \leq \lambda < 1$ is the temporal smoothing coefficient. A larger λ produces slower changes and favors stability, whereas a smaller value responds more rapidly to recent observations.

The update can be implemented with only a few scalar variables. In contrast with a predictor that estimates a complete next-state distribution, the timer maintains a smoothed representation of recent idle behavior.

C. Context-Aware Extension

Observed idle duration alone cannot fully represent application requirements. The framework therefore permits context to modify the energy-responsiveness preference, consistent with context-aware mobile scheduling and QoS-aware power-management designs [4], [9]. For example, an AR/VR application can increase the weight placed on delay, while night-time sensing can increase the emphasis on energy conservation.

We formalize this trade-off through

$$J(T) = w_E E(T) + w_D D(T), \quad (2)$$

where $E(T)$ denotes energy consumption and $D(T)$ denotes average response delay under threshold T . The coefficients w_E and w_D reflect the active policy objective and are distinct from the temporal smoothing coefficient λ . Table II shows representative settings used to explain three operating modes.

The weighting mechanism is intentionally simple. It provides a way to react to coarse application context without requiring a learned policy.

D. Algorithm

Algorithm 2 summarizes the control flow in compact form.

E. State-Machine Interpretation and Event Semantics

The framework can also be viewed as a three-state controller with *active*, *idle-wait*, and *low-power* states. After an event is serviced, the system enters the idle-wait state and arms the current threshold T . A new event arriving before expiry cancels the transition and produces a new idle observation. If the

```

on_event():
    now = now_ms()
    idle = now - last_event
    last_event = now

    if context == ARVR:
        w_D += delta
        w_E -= delta
    else if context == NIGHT:
        w_E += delta
        w_D -= delta

    T = lambda*T + (1-lambda)*idle

main_loop():
    while true:
        if idle_ms() >= T:
            suspend()
        if event_available():
            resume_if_needed()
            on_event()

```

Fig. 2. Conceptual pseudocode of the adaptive timer.

threshold expires, the Power Controller requests a low-power state. The next eligible user, sensor, or network event resumes the device and closes the current idle sample. This formulation separates *policy* (how T is selected) from *mechanism* (how the platform enters or exits low power), which is useful for heterogeneous IoT systems whose suspend primitives differ across operating systems and microcontrollers.

To avoid pathological behavior, a practical deployment can constrain the adaptive threshold to a platform-defined interval

$$T_{min} \leq T \leq T_{max}, \quad (3)$$

where T_{min} protects latency-sensitive applications from excessively frequent transitions and T_{max} limits idle energy waste. The implementation emphasizes lightweight adaptation rather than a learned transition policy; therefore, such bounds can be enforced with constant-time scalar operations and without changing the kernel interface.

F. Convergence and Responsiveness of the Update Rule

Equation (1) is an exponentially weighted update. For a stationary sequence with constant idle interval t^* , repeated application gives

$$T_k - t^* = \lambda^k (T_0 - t^*), \quad (4)$$

so the threshold approaches the observed idle interval geometrically whenever $0 \leq \lambda < 1$. This simple property clarifies the role of λ : larger values suppress short-term variation but require more events to follow a workload shift, while smaller values react faster but track transient bursts more closely. The implementation therefore exposes a direct and interpretable responsiveness knob rather than hiding adaptation inside model parameters.

TABLE III
QUALITATIVE POLICY CHARACTERISTICS

Property	Baseline	Fixed	ML	UATimer
Online adaptation	–	No	Yes	Yes
Training required	–	No	Yes	No
Context hook	OS dep.	No	Optional	Yes
Per-event update	OS dep.	$O(1)$	inference	$O(1)$
Kernel change	No	No	No	No
Policy state	OS dep.	scalar	coefficients	<1 KB

For a non-stationary event stream, the same update can be interpreted as an exponentially weighted moving average of past idle intervals,

$$T_k = \lambda^k T_0 + (1 - \lambda) \sum_{i=0}^{k-1} \lambda^i t_{k-i}, \quad (5)$$

which gives recent observations greater influence. This interpretation is important for IoT deployments because bursty user interaction, periodic sensing, and long unattended periods can occur on the same device. Context-aware changes to the energy–delay weights operate orthogonally to this temporal smoothing: λ controls how quickly the timer follows history, while (w_E, w_D) expresses the current application priority.

G. Policy Complexity and Deployment Properties

Table III summarizes the qualitative differences among the evaluated policies. The comparison is based on the implemented mechanisms rather than on additional performance measurements. The fixed timeout has the smallest control logic but cannot adapt. The linear-regression baseline adds prediction state and an inference path. UATimer retains constant-size scalar state and an $O(1)$ update per event while adapting online and accepting context input.

IV. IMPLEMENTATION

A. User-Space Integration

The implementation operates in user space and requires no kernel modification. On Android, the daemon registers PowerHAL-related callbacks to observe relevant activity and coordinate low-power transitions. On Linux, it uses `epoll`-based input hooks to detect events and a high-resolution timer to arm and disarm timeout decisions. Deployment changes are limited to user-space startup and manifest/init integration rather than kernel or driver patches.

The implementation size is below 500 lines of code, and the persistent policy state is below 1 KB. These properties are important for embedded deployment because they reduce the maintenance burden and avoid a model-management pipeline.

B. Cross-Architecture Portability

The control logic depends on timer APIs and user-space event hooks rather than processor-specific instructions. The implementation was evaluated primarily on ARM-based mobile platforms and additionally exercised on an Intel x86 Linux laptop and an ESP32-class RISC-V IoT board. This provides a portability check, not a full cross-architecture performance

evaluation. It indicates that the control logic can be transferred when equivalent event and suspend/resume interfaces are available; the present dataset does not contain energy or latency measurements for the additional x86 and RISC-V systems.

C. Lightweight ML Baseline

For comparison, a linear-regression scheduler predicts the next idle interval from the previous three idle intervals and a bias term. The model is trained by least squares on 10^3 samples and stores four scalar coefficients, requiring less than 64 B of model state. The reported per-event inference time is below 0.3 ms on a Cortex-A53 platform. This baseline was selected to represent a deliberately lightweight learning approach rather than a state-of-the-art neural or TinyML predictor. Consequently, the comparison supports only the claim that UATimer reaches a similar operating point to this specific predictor with simpler policy state; it must not be interpreted as a general comparison against learning-based power management.

V. EXPERIMENTAL METHODOLOGY

A. Platforms

The evaluation covers three representative device classes:

- an Android smartphone based on a Qualcomm Snapdragon platform with a 4000 mAh battery,
- a tablet based on an ARM Cortex-A73 platform with a 6000 mAh battery, and
- a Cortex-M4 IoT sensor node powered by a coin-cell battery.

Energy on the mobile devices was measured using a Monsoon power monitor, while an external meter was used for the IoT node. Event latency was obtained from timestamp logging. Wi-Fi was enabled and display brightness was fixed at 50% for the mobile-device experiments.

B. Workloads

Five workload classes were used:

- **Web browsing:** intermittent touch input mixed with background network activity;
- **Video playback:** continuous decoding with relatively little user input;
- **AI inference:** long-running on-device DNN execution;
- **AR/VR gaming:** 60-Hz rendering with frequent sensor and interaction events; and
- **IoT sensing:** periodic sensor sampling and wireless transmission.

C. Compared Policies

Four policies were compared:

- 1) **Baseline:** default OS behavior without additional timeout tuning;
- 2) **Fixed timeout:** a static timeout representative of conventional idle policies;
- 3) **ML scheduler:** the lightweight linear-regression predictor described above; and

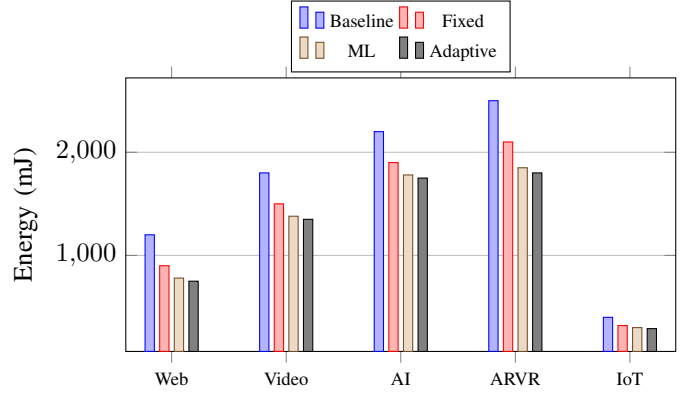


Fig. 3. Energy consumption for the evaluated workloads.

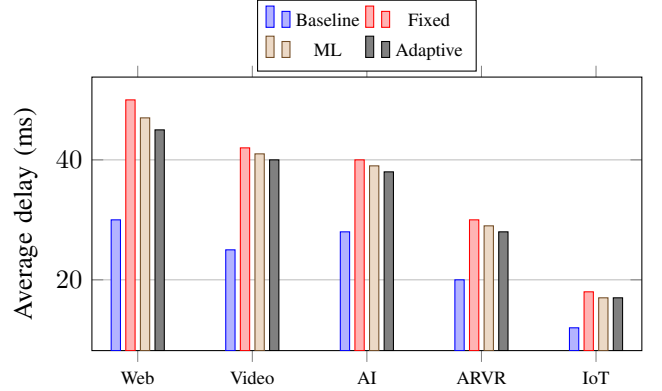


Fig. 4. Average event latency for the evaluated workloads.

- 4) **Adaptive timer:** the proposed user-aware context-enhanced policy.

Unless otherwise noted, the fixed-timeout configuration used 3 s for mobile workloads and 5 s for the IoT workload. Each experiment was repeated five times, and Table IV reports the arithmetic means available from those measurements. Because the archived dataset used to prepare this manuscript does not include the per-run samples, this version does not report reconstructed standard deviations, confidence intervals, or significance tests. This limitation is stated explicitly in Section VII.

VI. RESULTS

A. Energy and Latency

Table IV summarizes the representative measurements. The proposed adaptive timer consistently reduces energy relative to the baseline while keeping average event delay below 50 ms.

Across the five workload classes, direct calculation from Table IV gives baseline-relative energy reductions of 20.5–37.5% (mean 27.3% across the five listed workload means). Relative to the fixed-timeout policy, the reduction is 7.9–16.7%. The gap between the adaptive and ML schedulers is small in absolute energy (1.7–3.8% lower for UATimer in the listed measurements), so the result should be interpreted primarily as a complexity/deployability trade-off rather than a large performance advantage over prediction.

TABLE IV
REPRESENTATIVE EVALUATION RESULTS ACROSS WORKLOADS

Workload	Policy	Energy (mJ)	Delay (ms)	Frame drop (%)	Jitter (ms)
Web Browsing	Baseline	1200	30	–	–
	Fixed timeout	900	50	–	–
	ML scheduler	780	47	–	–
	Adaptive	750	45	–	–
Video Playback	Baseline	1800	25	–	–
	Fixed timeout	1500	42	–	–
	ML scheduler	1380	41	–	–
	Adaptive	1350	40	–	–
AI Inference	Baseline	2200	28	–	–
	Fixed timeout	1900	40	–	–
	ML scheduler	1780	39	–	–
	Adaptive	1750	38	–	–
AR/VR Gaming (60 Hz)	Baseline	2500	20	2.5	3.0
	Fixed timeout	2100	30	3.2	3.5
	ML scheduler	1850	29	1.8	2.7
	Adaptive	1800	28	0.5	1.2
IoT Sensing	Baseline	400	12	–	–
	Fixed timeout	320	18	–	–
	ML scheduler	300	17	–	–
	Adaptive	290	17	–	–

B. AR/VR Quality of Experience

The AR/VR workload illustrates the need to avoid overly aggressive energy saving. The fixed-timeout policy reduces energy but increases frame drops to 3.2% and jitter to 3.5 ms. The adaptive policy reduces energy from 2500 mJ to 1800 mJ while reporting only 0.5% frame drops and 1.2 ms jitter. These measurements indicate that context-dependent weighting can preserve interactive quality while still exploiting idle opportunities.

C. Comparison With the Lightweight ML Baseline

The adaptive timer slightly outperforms the linear-regression scheduler in the reported energy measurements while exhibiting similar latency. The main distinction is implementation complexity: the adaptive method requires only scalar state and a deterministic update rule, whereas the learning baseline requires a feature representation, trained coefficients, and inference logic. Although the chosen ML baseline is intentionally lightweight, the comparison demonstrates that a simple adaptive mechanism can capture much of the available benefit for the evaluated workloads.

D. Workload-Level Energy Reduction Derived From the Measurements

To make the workload dependence more explicit, Table V reports reductions computed directly from the energy values in Table IV. These percentages are arithmetic transformations of the reported measurements and do not represent additional experimental runs. Relative to the default baseline, the computed reduction is 20.5–37.5%; relative to the fixed-timeout policy, UATimer reduces measured energy by approximately 7.9–16.7% across the five workloads. Relative to the lightweight

TABLE V
ENERGY REDUCTION COMPUTED FROM TABLE IV

Workload	vs. Baseline (%)	vs. Fixed (%)	vs. ML (%)
Web browsing	37.5	16.7	3.8
Video playback	25.0	10.0	2.2
AI inference	20.5	7.9	1.7
AR/VR gaming	28.0	14.3	2.7
IoT sensing	27.5	9.4	3.3

ML scheduler, the additional reduction is smaller, approximately 1.7–3.8%, which supports the observation that a low-complexity adaptive rule captures much of the benefit available to the selected predictor.

The absolute measurements also show that the benefit is not tied to a single workload class. The adaptive policy has the lowest reported energy value in every row group of Table IV. At the same time, its latency remains close to that of the ML scheduler, and in the AR/VR case the frame-drop and jitter values are lower than those of the other evaluated policies. Because the experiment reports only five repetitions per condition and per-run samples are not available in the archived dataset, these workload-level comparisons should be interpreted as representative mean differences rather than universal effect sizes.

E. Energy–Latency Operating Point

The measurements illustrate why power management should be treated as a multi-objective problem. The baseline generally has the lowest event delay but consumes more energy, while fixed timeouts reduce energy at the cost of larger delay. UATimer occupies an intermediate operating point: it gives up some baseline responsiveness to gain energy efficiency, but its observed delay remains below 50 ms for all evaluated

workloads. For IoT deployments, this operating point can be shifted by changing context weights rather than replacing the control mechanism.

The AR/VR case is particularly informative. A purely energy-oriented policy can enter low-power states at times that disturb a 60-Hz rendering pipeline. By assigning greater weight to delay in latency-critical context, the timer can remain conservative while the application is active and become more aggressive when the context changes. Conversely, periodic sensing can tolerate longer thresholds and place more weight on energy conservation. The same runtime therefore supports heterogeneous device roles without maintaining a separate predictor for each workload class.

F. Failure Modes and Safe Integration

A deployable timer policy must account for cases in which suspending is undesirable even if the observed idle interval is long. Examples include non-deferrable background operations, protocol-critical communication windows, or device-specific transitions whose wake cost is high. The architecture addresses this at the integration boundary: platform or application context can inhibit a transition, select a latency-critical mode, or constrain T through Eq. (3). In this sense, UATimer is designed as a policy layer that complements rather than replaces existing platform safety and power-management controls.

The user-space placement also limits the failure domain. If the daemon is disabled or cannot obtain the required event hooks, the underlying OS policy remains available. This property is useful during staged deployment because the adaptive component can be enabled per product or workload without introducing an ISA-specific kernel patch. The trade-off is that user-space observation may not expose every device-level event; deeper component coordination would require additional platform interfaces.

G. Validity Considerations

Three considerations bound the interpretation of the present evaluation. *Internal validity* depends on stable measurement conditions; Wi-Fi state and display brightness were fixed and each experiment was repeated five times, but uncontrolled background activity can still affect mobile energy measurements. *Construct validity* is limited by the chosen metrics: total energy and average event latency capture the primary trade-off, while component-level residency, tail latency, and battery-lifetime projections are not reported. *External validity* is limited by the three principal evaluation platforms, even though the user-space daemon was additionally exercised on x86 Linux and an ESP32-class RISC-V board. These considerations motivate broader measurements but do not change the reported means.

H. Evidence Boundaries and Claims Supported by the Present Data

The available measurements support four bounded conclusions. First, the adaptive policy has the lowest listed energy mean among the four evaluated policies for each of the five

workloads. Second, average event delay for the adaptive policy remains below 50 ms in all listed workloads. Third, the reported AR/VR measurements show lower frame-drop and jitter values for the adaptive policy than for the three comparison policies. Fourth, the implementation is described as a small user-space component without kernel modification. The present data do *not* establish superiority over modern TinyML policies, statistical significance across a large sample, radio-specific IoT energy behavior, or general cross-architecture energy gains. We state these boundaries explicitly to keep the claims aligned with the available measurements.

VII. DISCUSSION

A. Why the Method Helps

The main benefit of the framework is not the timer itself but the continual alignment of timeout duration with observed behavior. A fixed timeout assumes a stationary environment, while interactive systems are inherently non-stationary. Web activity alternates between bursts and pauses, sensing workloads can become sparse, and latency-sensitive applications can temporarily dominate the system. The exponential update in (1) allows the threshold to follow these changes without a separate prediction model.

B. Workload Dependence

The magnitude of the gain depends on the availability of idle intervals. Video playback creates a relatively continuous decode pipeline and therefore provides fewer opportunities for suspend/resume optimization. More generally, prior measurements show that the energy impact of power-state decisions is highly workload- and interface-dependent [1], [5]. In contrast, web browsing and periodic sensing contain more exploitable idle gaps. This explains why the margin between the ML and adaptive schemes is narrow in some workloads: once the available idle opportunities are captured, additional prediction complexity has limited room to improve total energy.

C. Practical Deployment

A key property is that deployment does not require kernel patches. In production environments, kernel modifications can increase validation cost and complicate updates across device variants. A user-space approach can be deployed selectively, disabled when necessary, and updated independently of low-level drivers. The resulting design is therefore suitable for experimental integration in mobile-edge and IoT products where maintainability is as important as algorithmic efficiency.

D. Limitations

The current evaluation has several limitations. First, the device set is representative rather than exhaustive, and only one principal IoT sensing platform contributes quantitative results. Second, the learning comparison uses a simple linear-regression model; stronger TinyML predictors would provide a more demanding baseline. Third, the present results focus on device-level energy and event latency and do not provide a component-level breakdown across CPU, GPU, radio, display,

and memory. Fourth, only five repetitions were performed per condition, and the archived dataset available for this manuscript does not include per-run samples for retrospective uncertainty analysis. Finally, the context policy is rule-based and depends on manually defined application classes.

These limitations identify the highest-priority validation work for a subsequent experimental revision: evaluation on additional MCU and RISC-V boards, radio-specific experiments, battery-lifetime measurements, stronger TinyML baselines, component-level energy accounting, and a larger number of independent repetitions. Such extensions would strengthen generality without changing the core design.

E. Reproducibility

The implementation and evaluation scripts will be archived with the project repository at <https://github.com/leemgs/uatimer>. Reproducibility is particularly important for lightweight power-management mechanisms because small changes in timeout parameters, device state, and workload timing can affect measured energy savings.

VIII. CONCLUSION

This paper presented a user-aware adaptive timer framework for energy-efficient and responsive power management in heterogeneous IoT and mobile-edge systems. The framework dynamically updates the idle threshold from observed activity, uses lightweight context to shift energy-latency priorities, and operates entirely in user space without kernel modification. The implementation requires fewer than 500 lines of code and less than 1 KB of state. Across the reported browsing, video, AI, AR/VR, and sensing measurements, the adaptive mechanism reduces energy by 20.5–37.5% relative to the default baseline and 7.9–16.7% relative to the fixed-timeout policy while maintaining average event latency below 50 ms. It also reports favorable AR/VR quality indicators and energy values close to the deliberately lightweight linear-regression baseline, with simpler policy state. These characteristics make the design a practical middle ground between static timeout policies and heavier predictive schedulers for resource-constrained and interactive IoT platforms.

Future work will evaluate larger device populations, additional wireless protocols and edge workloads, stronger TinyML baselines, component-level energy behavior, and

more independent repetitions. Another direction is joint management of CPU, GPU, display, and radio power states while preserving the same low-overhead user-space design principle.

ACKNOWLEDGMENT

OpenAI ChatGPT was used to assist with drafting and restructuring portions of the Introduction, Background, Discussion, and language throughout the manuscript. The author reviewed and edited the generated text and takes responsibility for the final content. The AI system was not used to generate, alter, or fabricate experimental measurements or reported results.

REFERENCES

- [1] A. Carroll and G. Heiser, "An analysis of power consumption in a smartphone," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2010.
- [2] Y. Kim and J. Kim, "Personalized Diapause: Reducing radio energy consumption of smartphones by network-context aware dormancy predictions," in *Proc. Workshop Power-Aware Computing and Systems (HotPower)*, 2012.
- [3] I. Pefkianakis, J. Chandrashekar, and H. Lundgren, "User-driven idle energy save for 802.11x mobile devices," in *Proc. IEEE Int. Conf. Mobile Ad Hoc Sensor Syst. (MASS)*, 2014, pp. 282–290, doi: 10.1109/MASS.2014.50.
- [4] J. Lee, K. Lee, E. Jeong, J. Jo, and N. B. Shroff, "CAS: Context-aware background application scheduling in interactive mobile systems," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1013–1029, May 2017, doi: 10.1109/JSAC.2017.2676918.
- [5] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zarella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1224–1242, Jan. 2021, doi: 10.1109/JIOT.2020.3013949.
- [6] J. Sheth, C. Miremadi, A. Dezfouli, and B. Dezfouli, "EAPS: Edge-assisted predictive sleep scheduling for 802.11 IoT stations," *IEEE Syst. J.*, vol. 16, no. 1, pp. 591–602, Mar. 2022, doi: 10.1109/JSYST.2021.3063922.
- [7] B. Safaei, A. M. Hosseini Monazzah, and A. Ejlali, "ELITE: An elaborated cross-layer RPL objective function to achieve energy efficiency in Internet-of-Things devices," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1169–1182, Jan. 2021, doi: 10.1109/JIOT.2020.3011968.
- [8] H. Ko, H. Choi, and S. Pack, "Restoration-aware sleep scheduling framework in energy harvesting Internet of Things: A deep reinforcement learning approach," *IEEE Trans. Sustainable Comput.*, vol. 10, no. 1, pp. 190–198, Jan./Feb. 2025, doi: 10.1109/TSUSC.2024.3442918.
- [9] Q. Sang, J. Yan, R. Xie, C. Hu, K. Suo, and D. Cheng, "QoS-aware power management via scheduling and governing co-optimization on mobile devices," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13654–13669, Dec. 2024, doi: 10.1109/TMC.2024.3438267.
- [10] D. E. Ruiz-Guirola, S. Montejo-Sanchez, I. Leyva-Mayorga, Z. Han, P. Popovski, and O. L. A. Lopez, "Energy management and wakeup for IoT networks powered by energy harvesting," *IEEE Internet Things J.*, vol. 13, no. 9, pp. 18574–18591, May 2026, doi: 10.1109/JIOT.2026.3664201.